Camera Calibration Flow & Mathematical Formulas

1 Calibration Flow Overview

This document outlines the pipeline used to calibrate the multi-camera system, mapping 2D pixel coordinates from video streams to 3D real-world coordinates. The flow consists of two primary stages: extracting calibration points using `validate.py` and finding the optimal camera angles using a global minimization script.

1.1 Stage 1: Data Extraction (`validate.py`)

The `validate.py` script initializes a `MultiCameraPipeline` using the YOLOv8 object detector (`VideoObjectDetector`). Its primary purpose in the calibration flow is to process static images or video frames from specific camera views and extract the (u, v) pixel coordinates of known objects or reference points in the frame.

Inputs:

- Camera parameters (Height $H = 7.0$m, Intrinsic matrix K)
- Video/Image files representing the camera view
- Geographical coordinates (Base Lat/Lon)

Outputs: Detected bounding boxes or tracking points providing the u (horizontal) and v (vertical) pixel coordinates for reference ground points.

1.2 Stage 2: Angle Optimization (Minimization Script)

Once the (u, v) pixel coordinates are paired with their known real-world (X, Z) coordinates relative to the camera pole, the minimization script calculates the exact Pitch and Yaw of the camera. It uses the `scipy.optimize.minimize` function (using the Nelder-Mead method) to find the angles that produce the lowest average distance error across all calibration points.

Outputs:

- Global optimal pitch and yaw (in degrees).
- A `calibration_errors.csv` file detailing the true vs. predicted coordinates and the total error per point.

2 Mathematical Model and Formulas

The calibration relies on pinhole camera geometry, 3D rotation matrices, and ray-plane intersection math.

2.1 Camera Intrinsic Matrix (K)

The intrinsic matrix maps 3D camera coordinates to 2D pixel coordinates. For the given 2560x1920 wide-angle camera, the focal lengths (f_x, f_y) and optical centers (c_x, c_y) are defined as:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1325.4 & 0 & 1280.0 \\ 0 & 1325.4 & 960.0 \\ 0 & 0 & 1 \end{bmatrix}$$

2.2 2D Pixel to 3D Camera Ray

To determine where a pixel points in the real world, the (u, v) pixel coordinates are converted into a normalized 3D directional ray $(\vec{r}_{cam})$ originating from the camera lens:

$$\vec{r}_{cam} = \begin{bmatrix} \frac{u - c_x}{f_x} \\ \frac{v - c_y}{f_y} \\ 1 \end{bmatrix}$$

2.3 Rotation Matrices (Pitch and Yaw)

The camera ray must be rotated to match the real-world orientation.

- **Pitch (θ_x):** Tilt up/down (rotation around the X-axis).
- **Yaw (θ_y):** Pan left/right (rotation around the Y-axis).

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta_x) & -\sin(\theta_x) \\ 0 & \sin(\theta_x) & \cos(\theta_x) \end{bmatrix}$$

$$R_y = \begin{bmatrix} \cos(\theta_y) & 0 & \sin(\theta_y) \\ 0 & 1 & 0 \\ -\sin(\theta_y) & 0 & \cos(\theta_y) \end{bmatrix}$$

The combined rotation matrix R is applied to the camera ray to get the world-oriented directional vector $(\vec{r}_{world})$:

$$R = R_y R_x$$

$$\vec{r}_{world} = \begin{bmatrix} d_x \\ d_y \\ d_z \end{bmatrix} = R \cdot \vec{r}_{cam}$$

2.4 Ground Plane Intersection

The camera is mounted at height $H = 7.0$ meters. Assuming the ground is completely flat, we need to find the intersection of the 3D ray with the ground plane.

First, calculate a scaling factor t based on the camera's height and the vertical component of the ray (d_y). (Note: If $d_y \leq 0$, the ray is pointing at or above the horizon and will not intersect the ground).

$$t = \frac{H}{d_y}$$

Multiply the horizontal components of the ray by t to find the predicted real-world coordinates (X_{pred}, Z_{pred}):

$$X_{pred} = t \cdot d_x$$
$$Z_{pred} = t \cdot d_z$$

2.5 Error Optimization Function

To find the perfect pitch and yaw, the algorithm compares the predicted coordinates against the known true coordinates (X_{true}, Z_{true}). The Euclidean distance error for a single calibration point i is:

$$E_i = \sqrt{(X_{pred,i} - X_{true,i})^2 + (Z_{pred,i} - Z_{true,i})^2}$$

The global objective function minimizes the **average error** across all N calibration points:

$$\text{Cost}(\theta_x, \theta_y) = \frac{1}{N} \sum_{i=1}^{N} E_i$$

The script starts with an initial guess of Pitch = -40° and Yaw = -30°, iteratively adjusting the angles to drive this cost function as close to zero as possible.